Capture the Flag 4: The Student List

Goal: Build a scrollable student directory using Dart Lists and `ListView.builder`, replacing manually duplicated widgets with dynamically generated UI.

Instructor Note

These activities require your own problem-solving and research skills.

DO NOT use AI to generate the code for you.

You may use AI to:

- Research what a Dart/Flutter feature does
- Understand syntax
- Explain an error (not fix your error)
- Clarify documentation

You may NOT ask AI to generate the solution/code for a flag.

🚩 Flag 0 — Create Your Feature Branch

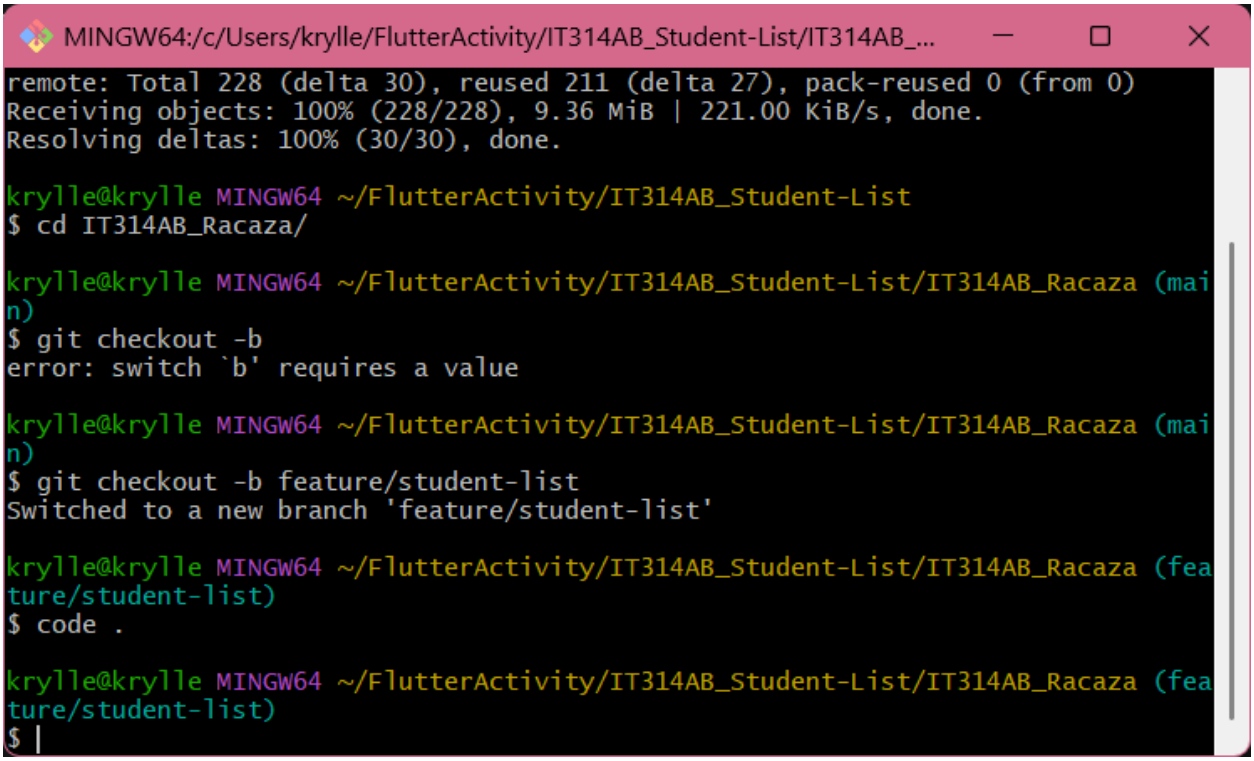
Objective: Create a feature branch before building your student list.

Create:

`feature/student-list`

Screenshot:

- Terminal showing your current branch



🚩 Flag 1 — Build Your Student Data

Objective: Store multiple students inside a Dart `List`.

Research what a Dart `List` is and why it is useful when handling multiple pieces of similar data.

Instead of creating one student at a time with separate variables, create a collection that stores at least five students.

Each student should contain information such as:

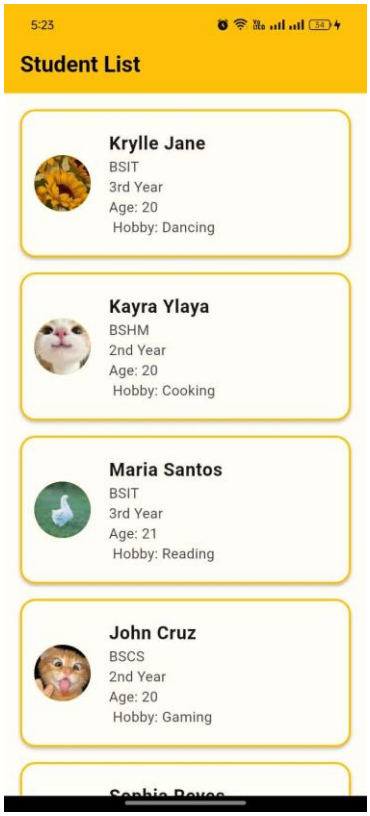
- Image
- Name
- Course
- Year Level
- Age
- Hobby

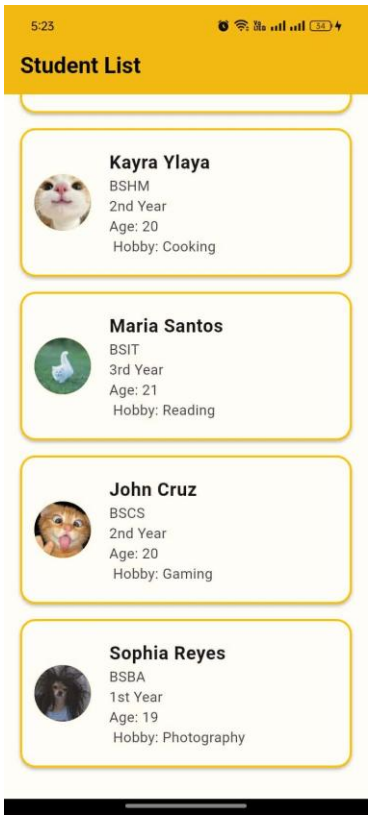
Your data should exist separately from your UI widgets.

Screenshot:

Code:

- Your student list
- At least five student entries






```dart
List<Map<String, dynamic>> students = [
  {
    'image': 'assets/profile1.png',
    'name': 'Krylle Jane',
    'course': 'BSIT',
    'year': '3rd Year',
    'age': 20,
    'hobby': 'Dancing',
  },
  {
    'image': 'assets/profile2.png',
    'name': 'Kayra Ylaya',
    'course': 'BSHM',
    'year': '2nd Year',
    'age': 20,
    'hobby': 'Cooking',
  },
  {
    'image': 'assets/profile3.png',
    'name': 'Maria Santos',
    'course': 'BSIT',
    'year': '3rd Year',
    'age': 21,
    'hobby': 'Reading',
  },
```

```dart
  {
    'image': 'assets/profile4.png',
    'name': 'John Cruz',
    'course': 'BSCS',
    'year': '2nd Year',
    'age': 20,
    'hobby': 'Gaming',
  },
  {
    'image': 'assets/profile5.png',
    'name': 'Sophia Reyes',
    'course': 'BSBA',
    'year': '1st Year',
    'age': 19,
    'hobby': 'Photography',
  },
];
```

🚩 Flag 2 — The Copy-Paste Trap

You already know how to create a Student Card.

You created five student cards manually.

You duplicated the same card design until all five students appear.

Stop.

Imagine having:

- 50 students.
- 500 students.
- 5,000 students.

Would copying the same widget over and over still be practical?

This flag is intentionally making you feel the problem before solving it.

Screenshot:

- Your five manually created student cards (do not change your past code yet)

```
body: ListView(
  padding: const EdgeInsets.all(16),

  children: [

    Card(
      elevation: 2,
      margin: const EdgeInsets.only(bottom: 15),

      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(15),

        side: const BorderSide(
          color: Color(0xFFFFC107),
          width: 2,
        ), // BorderSide
      ), // RoundedRectangleBorder

      color: const Color(0xFFFFFDF5),

      child: ListTile(
        contentPadding: const EdgeInsets.all(12),

        leading: CircleAvatar(
          radius: 30,
          backgroundColor: const Color(0xFFFFC107),

          backgroundImage: AssetImage(image1),
        ), // CircleAvatar

        title: Text(
          name1,
          style: const TextStyle(
            fontSize: 18,
            fontWeight: FontWeight.bold,
          ), // TextStyle
        ), // Text
```

```
        subtitle: Text(
          '$course1\n'
          '$year1\n'
          'Age: $age1\n'
          'Hobby: $hobby1',
        ), // Text
      ), // ListTile
    ), // Card

    Card(
      elevation: 2,
      margin: const EdgeInsets.only(bottom: 15),

      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(15),

        side: const BorderSide(
          color: Color(0xFFFFC107),
          width: 2,
        ), // BorderSide
      ), // RoundedRectangleBorder

      color: const Color(0xFFFFFDF5),

      child: ListTile(
        contentPadding: const EdgeInsets.all(12),

        leading: CircleAvatar(
          radius: 30,
          backgroundColor: const Color(0xFFFFC107),

          backgroundImage: AssetImage(image2),
        ), // CircleAvatar

        title: Text(
          name2,
          style: const TextStyle(
            fontSize: 18,
            fontWeight: FontWeight.bold,
```

```
        title: Text(
          name3,
          style: const TextStyle(
            fontSize: 18,
            fontWeight: FontWeight.bold,
          ), // TextStyle
        ), // Text

        subtitle: Text(
          '$course3\n'
          '$year3\n'
          'Age: $age3\n'
          'Hobby: $hobby3',
        ), // Text
      ), // ListTile
    ), // Card

    Card(
      elevation: 2,
      margin: const EdgeInsets.only(bottom: 15),

      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(15),

        side: const BorderSide(
          color: Color(0xFFFFC107),
          width: 2,
        ), // BorderSide
      ), // RoundedRectangleBorder

      color: const Color(0xFFFFFDF5),

      child: ListTile(
        contentPadding: const EdgeInsets.all(12),

        leading: CircleAvatar(
          radius: 30,
          backgroundColor: const Color(0xFFFFC107),

          backgroundImage: AssetImage(image4),
```

```
  title: Text(
    name4,
    style: const TextStyle(
      fontSize: 18,
      fontWeight: FontWeight.bold,
    ), // TextStyle
  ), // Text

  subtitle: Text(
    '$course4\n'
    '$year4\n'
    'Age: $age4\n'
    'Hobby: $hobby4',
  ), // Text
 ), // ListTile
), // Card

ard(
 elevation: 2,
 margin: const EdgeInsets.only(bottom: 15),

 shape: RoundedRectangleBorder(
  borderRadius: BorderRadius.circular(15),

  side: const BorderSide(
    color: Color(0xFFFFC107),
    width: 2,
  ), // BorderSide
 ), // RoundedRectangleBorder

 color: const Color(0xFFFFFDF5),

 child: ListTile(
  contentPadding: const EdgeInsets.all(12),

  leading: CircleAvatar(
    radius: 30,
    backgroundColor: const Color(0xFFFFC107),
```

```
leading: CircleAvatar(
  radius: 30,
  backgroundColor: const Color(0xFFFFC107),

  backgroundImage: AssetImage(image5),
), // CircleAvatar

title: Text(
  name5,
  style: const TextStyle(
    fontSize: 18,
    fontWeight: FontWeight.bold,
  ), // TextStyle
), // Text

subtitle: Text(
  '$course5\n'
  '$year5\n'
  'Age: $age5\n'
  'Hobby: $hobby5',
), // Text
```

🚩 Flag 3 — Discover `ListView`

Objective: Make your student directory scrollable using `ListView`.

Research Flutter's `ListView`.

Replace your previous layout with a scrolling layout so users can browse through all of your student cards naturally.

Your application should now allow continuous vertical scrolling.

Test

The user should be able to:

- Scroll from the first student
- Continue scrolling
- Reach the last student without resizing the screen

Screenshot:

- `ListView` in your code
- Top of the application
- Bottom portion showing later students

```
body: ListView(
  padding: const EdgeInsets.all(16),
```

Student List



Krylle Jane
BSIT
3rd Year
Age: 20
Hobby: Dancing



Kayra Ylaya
BSHM
2nd Year
Age: 20
Hobby: Cooking



Maria Santos
BSIT
3rd Year
Age: 21
Hobby: Reading



John Cruz
BSCS
2nd Year
Age: 20
Hobby: Gaming

Sophia Reyes

Student List



Kayra Ylaya
BSHM
2nd Year
Age: 20
Hobby: Cooking



Maria Santos
BSIT
3rd Year
Age: 21
Hobby: Reading



John Cruz
BSCS
2nd Year
Age: 20
Hobby: Gaming



Sophia Reyes
BSBA
1st Year
Age: 19
Hobby: Photography

🚩 Flag 4 — Unlock `ListView.builder`

Objective: Generate student cards automatically using `ListView.builder`.

Research `ListView.builder`.

Instead of writing five separate Student Cards, generate them automatically from your student list.

Your UI should now create one card for each student in your list.

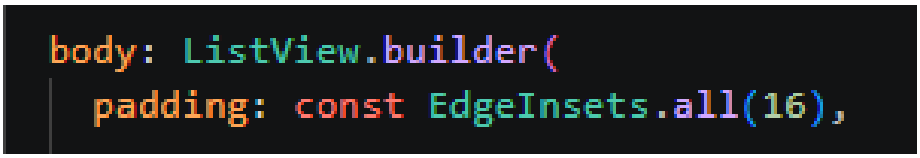
Test

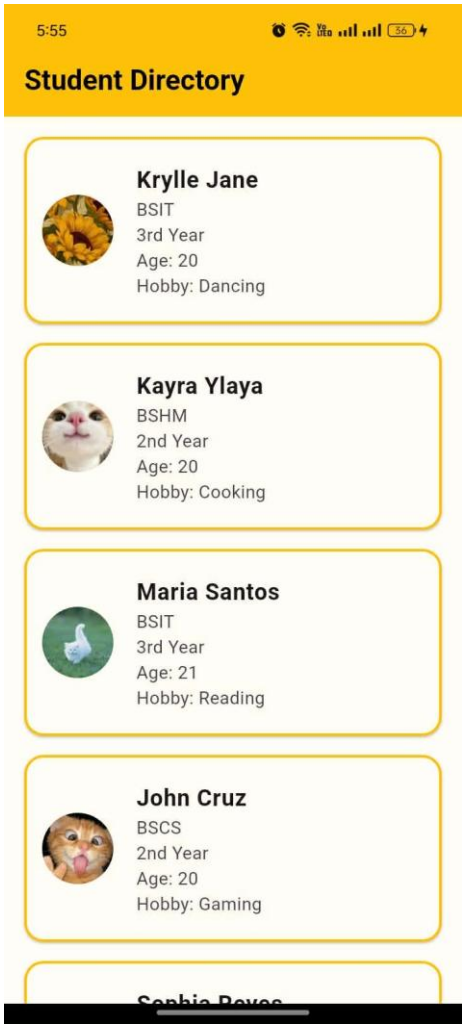
Add one new student to your list.

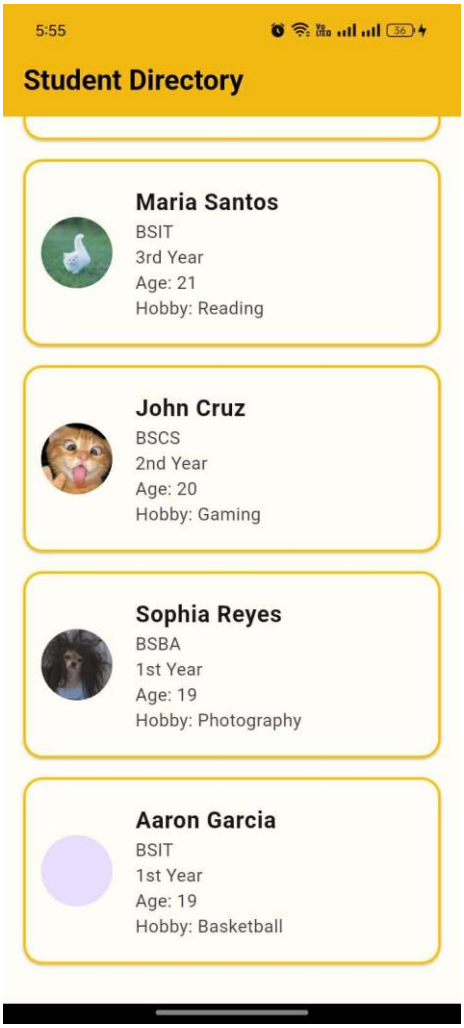
Without creating another card manually, your application should display the new student automatically.

Screenshot:

- `ListView.builder` in your code
- Student list
- Newly generated student appearing in the app








```dart
List<Map<String, dynamic>> students = [
  {
    'image': 'assets/profile1.png',
    'name': 'Krylle Jane',
    'course': 'BSIT',
    'year': '3rd Year',
    'age': 20,
    'hobby': 'Dancing',
  },
  {
    'image': 'assets/profile2.png',
    'name': 'Kayra Ylaya',
    'course': 'BSHM',
    'year': '2nd Year',
    'age': 20,
    'hobby': 'Cooking',
  },
  {
    'image': 'assets/profile3.png',
    'name': 'Maria Santos',
    'course': 'BSIT',
    'year': '3rd Year',
    'age': 21,
    'hobby': 'Reading',
  },
  {
    'image': 'assets/profile4.png',
    'name': 'John Cruz',
    'course': 'BSCS',
    'year': '2nd Year',
    'age': 20,
    'hobby': 'Gaming',
  },
```

```dart
  {
    'image': 'assets/profile5.png',
    'name': 'Sophia Reyes',
    'course': 'BSBA',
    'year': '1st Year',
    'age': 19,
    'hobby': 'Photography',
  },
  {
  'image': 'assets/profile6.png',
  'name': 'Aaron Garcia',
  'course': 'BSIT',
  'year': '1st Year',
  'age': 19,
  'hobby': 'Basketball',
},
];
```

🚩 Flag 5 — Every Student Deserves an ID

Objective: Expand your student records with additional information.

Real student directories contain more than just names.

Add at least three new fields to every student.

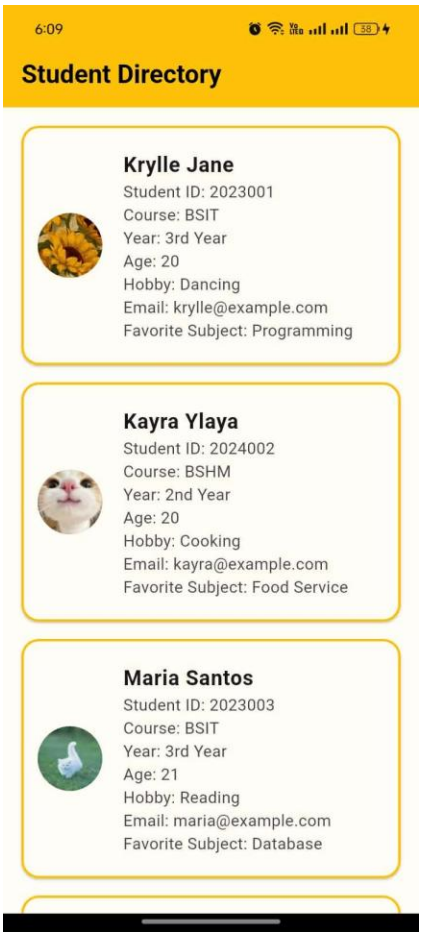
Examples:

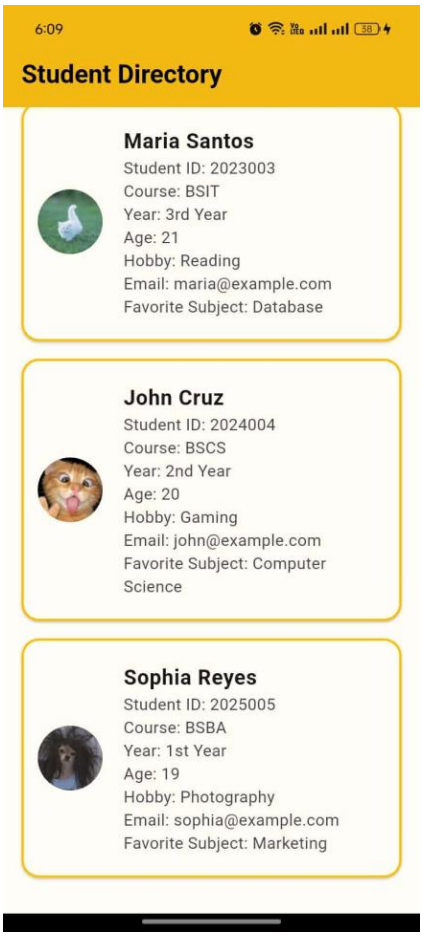
- Student ID
- Email
- Favorite Subject

Update your Student Card so the new information appears consistently across every generated card.

Screenshot:

- Updated student data
- Student Card displaying the additional fields






```dart
List<Map<String, dynamic>> students = [
  {
    'image': 'assets/profile1.png',
    'name': 'Krylle Jane',
    'course': 'BSIT',
    'year': '3rd Year',
    'age': 20,
    'hobby': 'Dancing',
    'studentId': '2023001',
    'email': 'krylle@example.com',
    'favoriteSubject': 'Programming',
  },
  {
    'image': 'assets/profile2.png',
    'name': 'Kayra Ylaya',
    'course': 'BSHM',
    'year': '2nd Year',
    'age': 20,
    'hobby': 'Cooking',
    'studentId': '2024002',
    'email': 'kayra@example.com',
    'favoriteSubject': 'Food Service',
  },
  {
    'image': 'assets/profile3.png',
    'name': 'Maria Santos',
    'course': 'BSIT',
    'year': '3rd Year',
    'age': 21,
    'hobby': 'Reading',
    'studentId': '2023003',
    'email': 'maria@example.com',
    'favoriteSubject': 'Database',
  },
```

```dart
  {
    'image': 'assets/profile4.png',
    'name': 'John Cruz',
    'course': 'BSCS',
    'year': '2nd Year',
    'age': 20,
    'hobby': 'Gaming',
    'studentId': '2024004',
    'email': 'john@example.com',
    'favoriteSubject': 'Computer Science',
  },
  {
    'image': 'assets/profile5.png',
    'name': 'Sophia Reyes',
    'course': 'BSBA',
    'year': '1st Year',
    'age': 19,
    'hobby': 'Photography',
    'studentId': '2025005',
    'email': 'sophia@example.com',
    'favoriteSubject': 'Marketing',
  },
];
```

🚩 Flag 6 — Empty Classroom

What happens when there are no students?

Objective: Handle an empty `List` gracefully.

Imagine your app just connected to a database.

The database returns:

[]

No students.

What should the user see?

Instead of displaying a broken or confusing screen, research how Flutter can display a different widget when a condition is met.

Create an empty state that clearly communicates that there are currently no students available.

Examples:

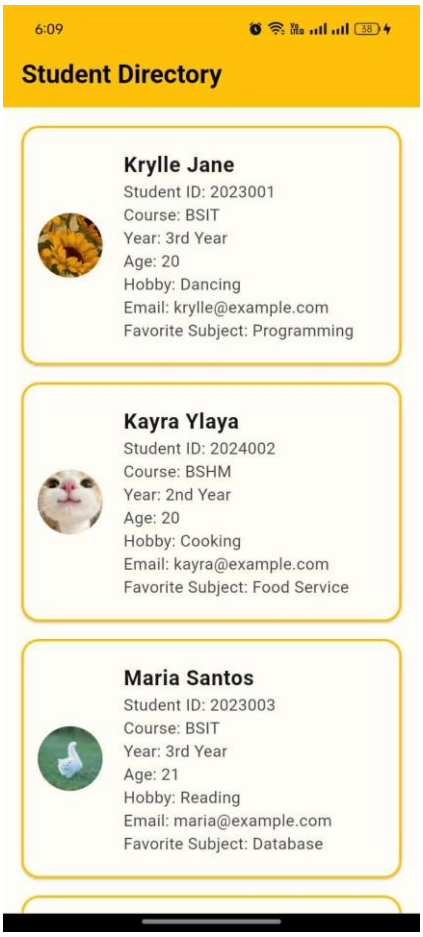
- "No students found."
- "Student list is empty."

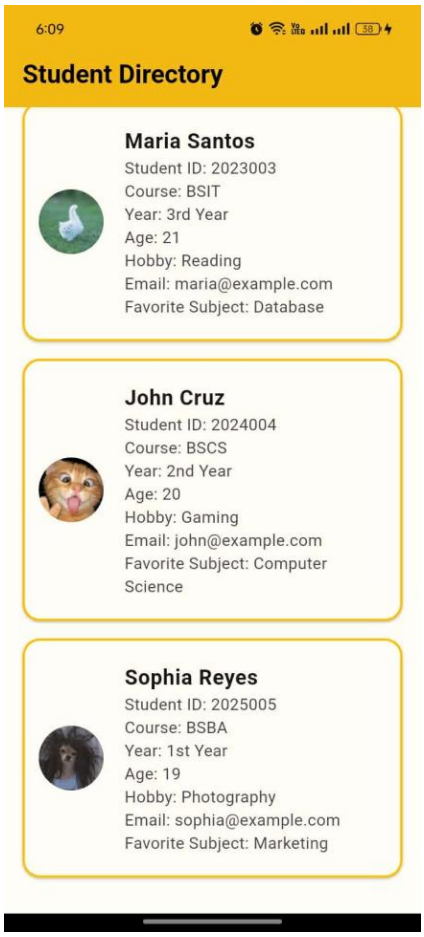
Test

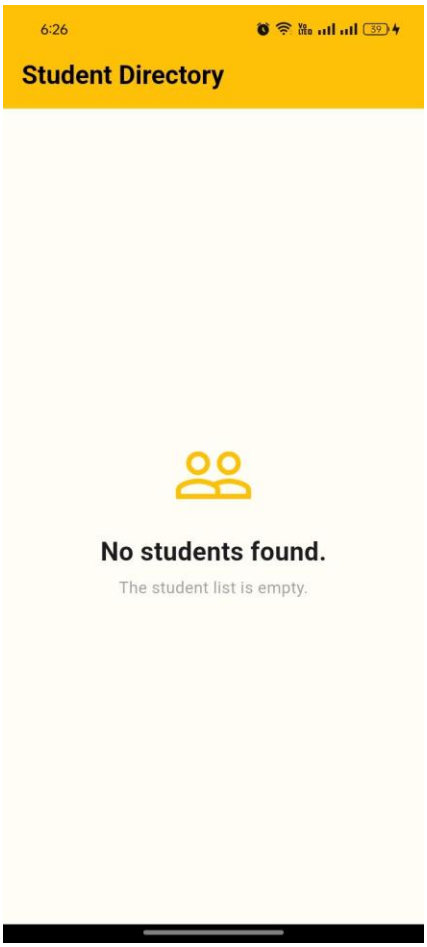
- Display your student list normally.
- Then temporarily make your list empty.
- Your application should display the empty state instead of student cards.

Screenshot:

- Empty student list
- Empty-state UI
- Application still functioning correctly








```
List<Map<String, dynamic>> students = [
];
```

🚩 Flag 7 — Sort the Roll Call

Objective: Change the order of your student list.

Class lists are rarely displayed randomly.

Research how Dart can rearrange items inside a `List`.

Sort your students alphabetically by name.

Test

Before sorting:

- Maria
- John
- Aaron
- Sophia

After sorting:

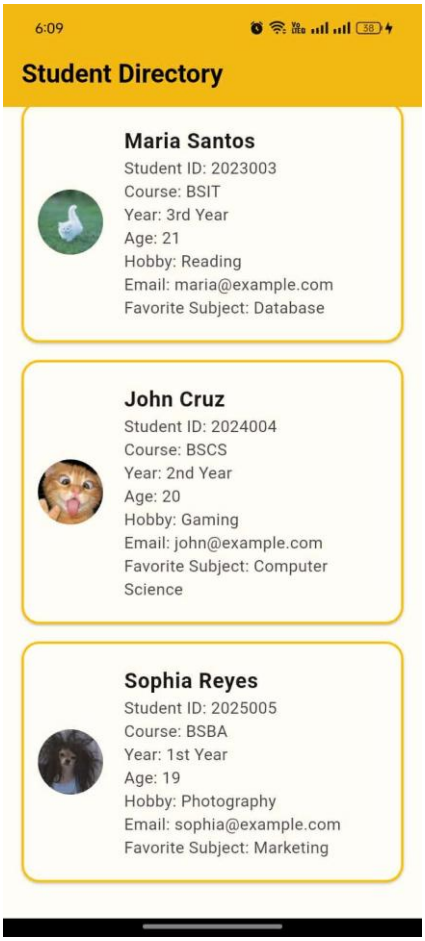
- Aaron
- John
- Maria
- Sophia

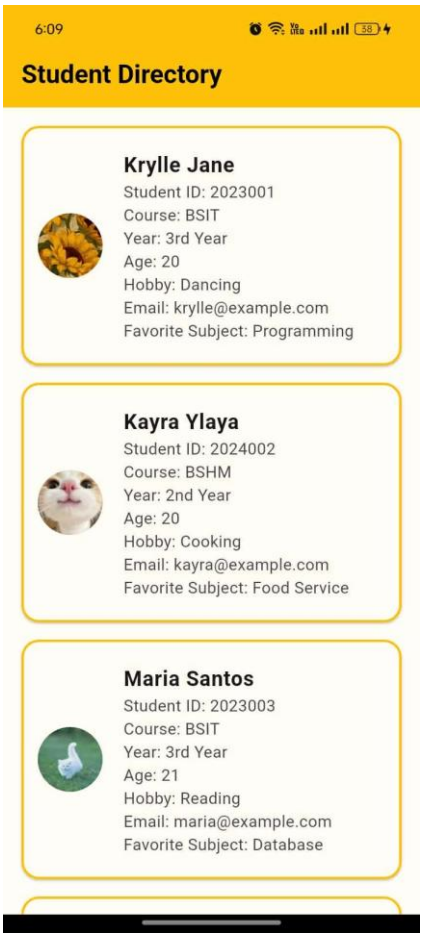
Your UI should immediately reflect the new order.

Screenshot:

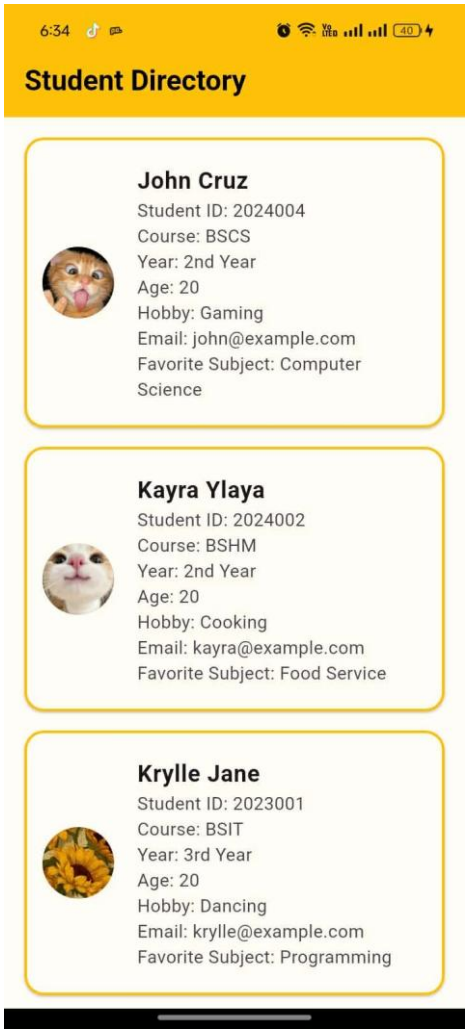
- Student list before or after sorting
- Application displaying the sorted order

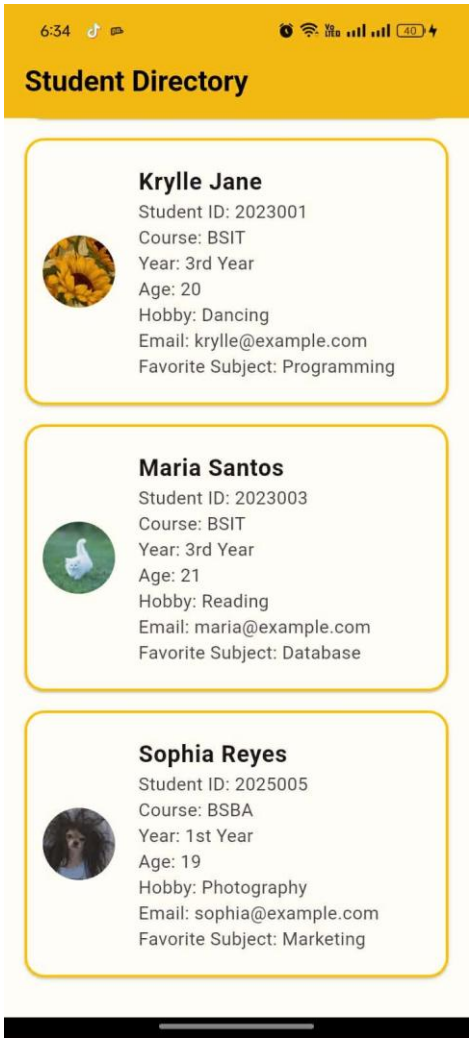
BEFORE:


6:09
Student Directory
Maria Santos
Student ID: 2023003
Course: BSIT
Year: 3rd Year
Age: 21
Hobby: Reading
Email: maria@example.com
Favorite Subject: Database
John Cruz
Student ID: 2024004
Course: BSCS
Year: 2nd Year
Age: 20
Hobby: Gaming
Email: john@example.com
Favorite Subject: Computer Science
Sophia Reyes
Student ID: 2025005
Course: BSBA
Year: 1st Year
Age: 19
Hobby: Photography
Email: sophia@example.com
Favorite Subject: Marketing


6:09
Student Directory
Krylle Jane
Student ID: 2023001
Course: BSIT
Year: 3rd Year
Age: 20
Hobby: Dancing
Email: krylle@example.com
Favorite Subject: Programming
Kayra Ylaya
Student ID: 2024002
Course: BSHM
Year: 2nd Year
Age: 20
Hobby: Cooking
Email: kayra@example.com
Favorite Subject: Food Service
Maria Santos
Student ID: 2023003
Course: BSIT
Year: 3rd Year
Age: 21
Hobby: Reading
Email: maria@example.com
Favorite Subject: Database

AFTER:


6:34
Student Directory
John Cruz
Student ID: 2024004
Course: BSCS
Year: 2nd Year
Age: 20
Hobby: Gaming
Email: john@example.com
Favorite Subject: Computer Science
Kayra Ylaya
Student ID: 2024002
Course: BSHM
Year: 2nd Year
Age: 20
Hobby: Cooking
Email: kayra@example.com
Favorite Subject: Food Service
Krylle Jane
Student ID: 2023001
Course: BSIT
Year: 3rd Year
Age: 20
Hobby: Dancing
Email: krylle@example.com
Favorite Subject: Programming


6:34
Student Directory
Krylle Jane
Student ID: 2023001
Course: BSIT
Year: 3rd Year
Age: 20
Hobby: Dancing
Email: krylle@example.com
Favorite Subject: Programming
Maria Santos
Student ID: 2023003
Course: BSIT
Year: 3rd Year
Age: 21
Hobby: Reading
Email: maria@example.com
Favorite Subject: Database
Sophia Reyes
Student ID: 2025005
Course: BSBA
Year: 1st Year
Age: 19
Hobby: Photography
Email: sophia@example.com
Favorite Subject: Marketing

🚩 Flag 8 — Save Your Progress

Objective: Commit and push your completed Student List CTF.

Use an appropriate commit message following proper Git standards.

Branch:

`feature/student-list`

Screenshot:

- Successful commit
- Current branch
- Terminal showing the commit

